ODSM v2 - Description Complete de l'Application

Version : ODSM v2 (application React/Next.js avec backend Prisma/SQLite)
Document genere automatiquement a partir de lanalyse du code source.

1. Objectif et valeur metier

ODSM v2 est une plateforme de gestion scolaire concue pour centraliser l'administration des etablissements educatifs. Elle vise a offrir aux directions, responsables financiers, secretaires, enseignants et superviseurs une solution integree pour gerer :
- les eleves et leurs dossiers scolaires,
- le personnel enseignant et administratif,
- les finances et paiements,
- les emplois du temps et la distribution des cours,
- les annees scolaires actives, archivees et en draft.

La solution repond a des besoins concrets de tracabilite, de securite, de separation des droits par role et de transition entre annees scolaires. Elle est structuree pour preserver les archives des annees scolaires anterieures, tout en simplifiant la migration vers une nouvelle annee scolaire.

2. Architecture structurelle

La couche frontend est construite avec Next.js 16 en mode App Router, utilisant React 19 et TypeScript. La structure de l'application repose sur un layout principal, des composants reutilisables, des providers de donnees et des routes dediees.

Les sections principales exposees par l'application sont :
- Dashboard,
- Eleves,
- Personnel,
- Finances,
- Locaux (classrooms),
- Horaires,
- Parametres,
- Portail enseignant,
- Activites et journal.

Chaque section est contenue dans un repertoire de page dedie sous src/app/(main), avec des pages enfants pour les activites specifiques, les evaluations, les bulletins et la configuration.

3. Architecture logique

L'application s'organise autour de trois axes logiques : l'identite utilisateur, la donnee scolaire et la gestion des annees scolaires.

Le flux logique cle est le suivant :

- Authentification et session : l'utilisateur se connecte via un systeme de session chifree.
- Chargement des donnees scolaires : le provider central SchoolDataProvider recupere le contexte de l'ecole, le parametrage de l'annee scolaire et les donnees metiers.
- Autorisation par role : la navigation et l'acces aux routes sont verouilles via une strategie de permissions centralisee.
- Supervision de l'annee scolaire : le systeme sait distinguer annee active, archive et draft, et gere les sauvegardes de parametres par annee.
- Mode enseignant : les enseignants recoivent un dataset reduit et filtre sur leurs classes et emplois du temps.
- Mode administrateur : acces global aux eleves, paiements, personnel, structure et parametres de l'ecole.
- La couche API expose des routes REST serverless au sein de src/app/api pour fournir les donnees dirigees par session, ecole et annee scolaire.

4. Architecture logicielle

Le noyau logiciel combine plusieurs modules :
- Components : UI reutilisables, controles Radix, themes, header, footer, dialogues et formulaires.
- Providers : SchoolDataProvider, StudentDisplayProvider et SessionGuard orchestrent le partage de l'etat applicatif.
- Lib : services utilitaires pour la base de donnees, l'authentification, le chiffrement, la licence, la gestion des droits et des annees scolaires.
- Types : definitions TypeScript centralisees pour Student, StaffProfile, SchoolData, FeeCategory, Course, etc.
- Prisma : objet relationnel pour SQLite avec un schema riche decrivant les ecoles, eleves, enseignants, paiements, classes, evaluations, resultats et parametres d'annee.

La logique de la donnee scolaire est implementee dans src/app/api/school/route.ts, qui alimente l'interface avec :
- la configuration de l'ecole,
- le calendrier et l'annee active,
- la structure des cours et des emplois du temps,
- les enseignants et leurs assignations,
- les eleves, les paiements et les tuteurs,
- la distinction entre mode enseignant et mode admin.

Ce module integre aussi des regles metiers specifiques, comme la filtration des paiements selon l'annee scolaire demandee, la restitution des archives et la securite des identifiants.

5. Modele de donnees

La base de donnees est geree via Prisma et SQLite. Le schema principal comprend les entites :
- School : ecole et configuration globale,
- Student : dossier eleve, annee scolaire, classe, niveau et parent/tuteur,
- StaffProfile : personnel enseignant et administratif, roles et assignations,
- Payment : operations financieres et details de paiement,
- Classroom : salles et groupes,
- Assessment, Period, StudentResult : evaluation et suivi academique,
- Guardian : tuteurs et relations eleves,
- AnnualDecision/AnnualClosure : gestion des decisions academiques et fermetures par annee,
- SchoolYearSettings : stockage de parametres par annee scolaire, permettant l'archivage et les drafts.

Ce schema met l'accent sur la separation des donnees par etablissement et par annee scolaire, tout en restant

compatible avec les anciens formats de configuration.

6. Technologies utilisees

Frontend : Next.js 16 (App Router), React 19, TypeScript, Tailwind CSS 4, shadcn UI, Radix UI, lucide-react et Sonner pour les notifications.
Backend / persistance : Prisma 7.8 avec SQLite, adaptateur @prisma/adapter-better-sqlite3, et des API serverless integrees dans le meme projet Next.js.
Authentification et securite : cookies HTTP-only chiffrés, gestion de session via src/lib/auth.ts, controle de licence et verification des droits par role.
Utilitaires : fonctions de normalisation des annees scolaires, generation de chemins de navigation, filtrage des enseignants, gestion des donnees de configuration et du fallback legacy.
Plateforme : application web Progressive Web App possible avec un service worker, et dependances Capacitor pretes pour l'emballage mobile si necessaire.

7. Organisation des fichiers

Principaux repertoires :
- src/app : routes et pages, organisation par espace metier et par role,
- src/components : composants UI, header, footer, dialogues et widgets,
- src/components/providers : providers d'etat applicatif et contextes partages,
- src/lib : services metiers, helpers utilitaires et integrations,
- src/types : definitions TypeScript du modele de donnees,
- prisma : schema Prisma et base SQLite.

Ce decoupage reflete une architecture moderne de type feature-oriented, ou chaque domaine metier est isole tout en restant connecte par des services transverses.

8. Valeur technique et points forts

ODSM v2 est concu pour fournir une gestion complete de letablissement scolaire en combinant :
- une interface utilisateur reactive et role-aware,
- une data layer centralisee et coherente,
- une prise en charge des annees scolaires actives, archivees et en draft,
- une securite applicative basee sur des sessions chifrees et des permissions de route,
- un rendu cote serveur hybride pour de bonnes performances et SEO,
- une base de code extensible permettant lintegration future de migrations dannee scolaire et de modules metiers avances.

Le resultat est une application adaptee aux etablissements qui souhaitent remplacer des processus papier ou des outils financiers disparates par une plateforme unifiee et traçable.